

Small Language Models on Three Edge Boards: Orange Pi Zero 3W, Raspberry Pi 5, and Arduino UNO Q

Marcelo Rovai · Orange Pi Zero 3W, Raspberry Pi 5, and Arduino UNO Q · September 2026

Benchmarks run on 2026-09-27 and 2026-09-28 with llama.cpp, Unsloth and OpenBMB GGUFs.

Hardware

Board	SoC / CPU	RAM	Storage	Cooling
Orange Pi Zero 3W	Allwinner A733: 2× Cortex-A76 @ 2.0 GHz + 6× Cortex-A55 @ 1.8 GHz	6 GB (5.7 GiB usable)	32 GB microSD	Heatsink + PWM fan
Raspberry Pi 5	BCM2712: 4× Cortex-A76 @ 2.4 GHz	8 GB	NVMe	Active cooler
Arduino UNO Q	Qualcomm QRB2210: 4× Cortex-A53 @ 2.0 GHz	4 GB (3.6 GiB usable)	eMMC	None

Method

- **Software.** llama.cpp commit 136887b on all three boards, built with `-DCMAKE_BUILD_TYPE=Release`. The Raspberry Pi and UNO Q builds also used `-DGGML_NATIVE=ON`. The Raspberry Pi numbers for Gemma 4 E2B and Qwen3.5 4B come from the article [Running Small Language Models on a Raspberry Pi 5](#), which used build 91d2fc387.
- **Models.** Same files on every board, verified by SHA-256 where they were already present:
 - unsloth/Qwen3.5-0.8B-MTP-GGUF: Qwen3.5-0.8B-UD-Q8_K_XL.gguf (Q8_0)
 - unsloth/Qwen3.5-2B-MTP-GGUF: Qwen3.5-2B-UD-Q4_K_XL.gguf
 - unsloth/Qwen3.5-4B-MTP-GGUF: Qwen3.5-4B-UD-Q4_K_XL.gguf
 - unsloth/gemma-4-E2B-it-qat-GGUF: gemma-4-E2B-it-qat-UD-Q4_K_XL.gguf plus the draft model mtp-gemma-4-E2B-it.gguf
 - openbmb/MiniCPM5-1B-GGUF and openbmb/MiniCPM5-2B-GGUF: Q4_K_M
 - Vision projectors: mmproj-F16.gguf from the same Unsloth repositories.
- **Raw throughput.** llama-bench -p 512 -n 128 -r 3.
- **Generation and MTP.** Same protocol as the Raspberry Pi article:
 - llama-server with `--ctx-size 8192 --parallel 1 --flash-attn on --reasoning off --reasoning-budget 0`;
 - /completion with the prompt "Explain photosynthesis in 300 words.", `n_predict 256`, and `cache_prompt: false`;
 - 3 runs per configuration.
 - Sampling: the Qwen models used temp 0.6, top-p 0.95, top-k 20, and min-p 0. Gemma used temp 1.0, top-p 0.95, and top-k 64.
 - MiniCPM5 used `ignore_eos`, because a raw prompt without the chat template sometimes ended after one token.
- **Thread configurations.** `-t` sets the generation threads (batch of 1 token). `-tb` sets the threads for multi-token batches, which include both the prompt and the MTP verify step.
- **Thermal state.**
 - Raspberry Pi: `vcgencmd get_throttled` returned `0x0` on every run, with a maximum of 77.9 °C.

- Orange Pi: reached 72–86 °C with its fan at full speed. The first passive trip point is 90 °C, and the CPU cooling devices stayed at state 0.
- UNO Q, with no cooling: the busy-core clock was sampled every second during each run and stayed at 2016 MHz, with a maximum of 76.8 °C.

1. Raw throughput (llama-bench, tokens/s, best thread count)

Model	Test	Orange Pi Zero 3W	Raspberry Pi 5	UNO Q
Qwen3.5 0.8B Q8	pp512	65.4 (–t 8)	83.7 (–t 4)	6.0 (–t 4)
	tg128	7.57 (–t 2)	8.57 (–t 1)	4.84 (–t 4)
Qwen3.5 2B Q4_K_XL	pp512	30.3 (–t 8)	41.3 (–t 4)	4.22 (–t 4)
	tg128	5.94 (–t 2)	6.69 (–t 3)	2.53 (–t 4)
MiniCPM5 1B Q4_K_M	pp512	65.9 (–t 8)	104.0 (–t 4)	10.1 (–t 4)
	tg128	15.3 (–t 2)	18.4 (–t 2)	5.86 (–t 4)
MiniCPM5 2B Q4_K_M	pp512	23.3 (–t 8)	36.2 (–t 4)	3.55 (–t 4)
	tg128	6.32 (–t 2)	7.24 (–t 2)	2.50 (–t 4)

The UNO Q result for Qwen3.5 2B (4.22 / 2.53 t/s) matches the earlier measurement on that board (4.22 / 2.55 t/s).

Two kinds of bottleneck

Qwen3.5 2B tg128	1 thread	2 threads	3 threads	4 threads
Raspberry Pi 5	4.41	6.66	6.69	5.77
UNO Q	0.74	1.31	1.91	2.53

- **The Raspberry Pi 5 and the Orange Pi are memory-bound.** Generation peaks at 2–3 threads and then drops. Multiplying tokens/s by the model size gives about 9–13 GB/s on the Pi 5 and 8–10.5 GB/s on the Orange Pi. The Raspberry Pi value matches the 9.3 GB/s measured in the article. The product is a little lower for Qwen3.5 than for MiniCPM5, because Qwen3.5's recurrent (Gated DeltaNet) layers add compute per token.
- **The UNO Q is compute-bound.** Generation scales almost linearly with the number of threads, up to all four cores. The Cortex-A53 is an in-order core without the INT8 dot-product instruction (`sdot`), so the arithmetic, not the memory, sets its speed.
- **On the Orange Pi, only the two A76 cores should generate.** When the six A55 cores join, the A76 cores wait for them. The best setting is `–t 2 –tb 8` with no affinity flags: the scheduler already places the two generation threads on the A76 cores, and 8 threads raise prompt processing by 35–60%.

2. Plain decode vs. best MTP (llama-server, tokens/s)

Model	Board	No MTP	Best MTP	Config	Gain
Qwen3.5 0.8B Q8	Orange Pi	7.44	7.66 ± 0.47	n=1, -t 4	+3% (noise)
	Raspberry Pi	8.04	8.42 ± 1.26	n=2, -t 3	+5% (noise)
	UNO Q	4.70	3.55 ± 0.08	n=1, -t 4	-24%
Qwen3.5 2B Q4_K_XL	Orange Pi	5.98	5.98 ± 0.15	n=3, -t 4	0%
	Raspberry Pi	6.62	8.06 ± 0.34	n=3, -t 3	+22%
	UNO Q	1.31 (-t 2)	0.98 ± 0.02	n=1, -t 2	-25% ¹
Gemma 4 E2B QAT	Orange Pi	6.99	8.73 ± 0.38	n=3, -t 4	+25%
	Raspberry Pi ²	9.03	13.06	n=2, -t 3	+45%
Qwen3.5 4B Q4_K_XL	Orange Pi	2.70	3.45 ± 0.11	n=3, -t 2 -tb 8	+28%
	Raspberry Pi ²	3.12	4.83	n=3, -t 4	+55%

¹ The UNO Q 2B MTP sweep was stopped after n=1, because the 0.8B sweep had already shown that MTP only slows this board down. ² From the Raspberry Pi article (llama.cpp 91d2fc387), same protocol.

On the Orange Pi, a verify batch of 4 tokens (n=3) is the only draft depth that pays off, for all four models. This supports the article's rule that n+1 should be a multiple of 4. On the UNO Q, MTP is slower at every depth: the board is compute-bound, so verifying several tokens costs almost as much as generating them one by one.

MiniCPM5 (no MTP heads), llama-server, tokens/s

Model	Orange Pi	Raspberry Pi 5	UNO Q
MiniCPM5 1B Q4_K_M	14.72	17.67	5.63
MiniCPM5 2B Q4_K_M	6.13	7.10	2.44

3. Image captioning

The same photo on every board (640×480), with "Describe this image in one paragraph.", max_tokens 128, temperature 0, --image-max-tokens 256, and 3 runs after a warm-up. The table shows the total time for the best thread setting.

Model	Orange Pi Zero 3W	Raspberry Pi 5	UNO Q
Qwen3.5 0.8B Q8	22.1 s	18.9 s	102.5 s
Gemma 4 E2B QAT	32.6 s	28.5 s	— (does not fit ³)
Qwen3.5 2B Q4_K_XL	45.2 s	38.9 s	283.5 s
Qwen3.5 4B Q4_K_XL	86.4 s	74.4 s	— (does not fit ³)

³ See the memory table below.

Where the time goes on the Orange Pi. This comes from one cold run with -t 2 -tb 8 and no image-token cap, so the image produced 300 tokens for Qwen and 130 for Gemma:

Model	Vision encoder	Image-token decode	Share spent in the encoder
Qwen3.5 0.8B	7.6 s	5.0 s	60%
Qwen3.5 2B	24.2 s	10.3 s	70%
Gemma 4 E2B	11.8 s	3.2 s	78%
Qwen3.5 4B	23.2 s	25.6 s	48%

The 2B and 4B projectors are nearly the same size (637 and 641 MiB), so their encoder times match. Only the language-model part grows with the model.

4. Memory (Orange Pi, measured)

Peak memory of `llama-server` after one real request, loaded with `--load-mode none` (no mmap) so every byte is counted once. Context was 4096 tokens for the 0.8B, 2B, and MiniCPM5 models, and 8192 for Gemma and Qwen 4B, as in the article.

Configuration	Peak memory	Fits in 4 GB (~3.0 GiB free)?
MiniCPM5 1B, text	0.79 GiB	✓
Qwen3.5 0.8B, text	1.30 GiB	✓
Qwen3.5 0.8B, vision	1.59 GiB	✓
MiniCPM5 2B, text	1.68 GiB	✓
Qwen3.5 2B, text	1.84 GiB	✓
Qwen3.5 2B, vision	2.58 GiB	✓ (tight)
Gemma 4 E2B, text + MTP	3.04 GiB	✗ (at the limit)
Gemma 4 E2B, vision	3.86 GiB	✗
Qwen3.5 4B, text + MTP	4.01 GiB	✗
Qwen3.5 4B, vision	4.44 GiB	✗

A 4 GB board runs everything up to 2B, including vision. The two main models of the Raspberry Pi article (Gemma 4 E2B with vision and Qwen3.5 4B) need 6 GB.

With the default mmap loading, the process RSS looks larger. `llama.cpp` repacks quantized weights into an ARM-optimized layout in anonymous memory, while the original file pages also stay resident until the kernel reclaims them. The no-mmap numbers above are the real requirement.

5. Long contexts and agents (Orange Pi)

KV cache per token (F16, computed from GGUF metadata)

Model	Architecture	KV per token	8K	32K
Gemma 4 E2B	sliding window + shared KV	6 KiB	48 MiB	192 MiB
Qwen3.5 0.8B / 2B	hybrid: attention in 6 of 25 layers	12 KiB	96 MiB	384 MiB
Qwen3.5 4B	hybrid: attention in 8 of 33 layers	32 KiB	256 MiB	1 GiB
MiniCPM5 1B	full attention, 24 layers	24 KiB	192 MiB	768 MiB
MiniCPM5 2B	full attention, 42 layers	42 KiB	336 MiB	1.3 GiB

The Qwen3.5 4B value at 8K matches the 256 MiB that `llama.cpp` reported in the article.

Speed vs. context depth (llama-bench -d, tokens/s)

Model	pp512 @ 0	@ 4K	@ 16K	tg128 @ 0	@ 4K	@ 16K
Gemma 4 E2B	35.7	19.8	7.9	7.20	5.02	2.84
Qwen3.5 2B	29.7	25.4	16.7	5.90	4.50	2.62
MiniCPM5 2B	22.5	12.3	4.9	6.33	2.22	0.74
Qwen3.5 4B	11.1	9.4	6.1	2.73	1.82	0.92

MiniCPM5 2B is the fastest 2B model with a short context, and the slowest by far with a long one. At 4K tokens of context, it generates less than half as fast as Qwen3.5 2B. Its full attention on all 42 layers costs more with every token in the context.

A three-turn agent with 24 tools

This test used a system prompt plus 24 tool definitions (about 4,000 tokens), 8K context, and `-t 2 -tb 8`:

Model	Turn 1 (prompt processed)	Turn 1 total	Turns 2–3 total
Gemma 4 E2B	3,637 tokens	2.3–3.0 min	8 s
Qwen3.5 2B	4,149 tokens	2.6 min	12 s
MiniCPM5 2B	3,965 tokens	4.0 min	17–26 s
Qwen3.5 4B	4,149 tokens	7.1 min	17–33 s

Prompt caching works with the default settings. From turn 2 on, only the new tokens are processed: 41–78 tokens instead of about 4,000. `--checkpoint-min-step 256` made no difference. The cost of an agent on these boards is the first turn, so keep the server running and keep the tool definitions short.

6. Price and value

Amazon US prices, checked on 2026-09-28, for board plus cooling. The Raspberry Pi 5 needs the official Active Cooler, sold separately. The Orange Pi Zero 3W ships with an aluminum heatsink and a cooling fan in the box.

Board	Board	Cooler	Total	Qwen3.5 2B tg128 (t/s)	t/s per \$100	2B caption	Runs
Raspberry Pi 5 8GB	\$200.00	\$10.95	\$210.95	6.69	3.2	39 s	everything tested
Raspberry Pi 5 4GB	\$126.49	\$10.95	\$137.44	6.69	4.9	39 s	up to 2B, including vision ¹
Orange Pi Zero 3W 6GB	\$84.99	included ³	\$84.99	5.94	7.0	45 s	everything tested
Orange Pi Zero 3W 4GB	\$73.99	included ³	\$73.99	5.94	8.0	45 s	up to 2B, including vision ¹
Arduino UNO Q 4GB	\$79.00	— ²	\$79.00	2.53	3.2	284 s	up to 2B, including vision

¹ Same 4 GB limit as the UNO Q (see section 4). The 4 GB Raspberry Pi 5 and Orange Pi were not tested; their speed is assumed to equal the 8 GB and 6 GB boards, since they use the same SoC and memory type. ² The UNO Q ran every test without a heatsink or fan and without throttling. ³ The Amazon listing (sold by Orange Pi Official)

lists "Orange Pi Zero 3W Single Board Computer, Cooling Fan" in the box. Our board also came with the aluminum heatsink shown in the photos, and all tests used this heatsink-and-fan set.

- **Amazon prices for the Raspberry Pi 5 are well above the official list price.** The December 2025 list prices were \$70 (4 GB) and \$95 (8 GB) ([Raspberry Pi news](#)), but the Amazon listings came from third-party sellers at \$126.49 and \$200. The Orange Pi and UNO Q listings were sold by Orange Pi Official and by Arduino.
- **The Orange Pi Zero 3W gives the most speed per dollar.** It delivers 89% of the Raspberry Pi 5's generation speed at 40% of the price of the 8 GB Pi 5 with its cooler (at Amazon prices).
- **The 6 GB Orange Pi is the cheapest board that runs everything tested**, including Gemma 4 E2B with vision and Qwen3.5 4B.
- **The UNO Q costs about the same as the 4 GB Orange Pi, but is 2.3× slower at generation.** It makes sense when its microcontroller side (the STM32) matters, or when text-only answers of up to 2B are enough.

7. Takeaways

1. **The Raspberry Pi 5 is the fastest board.** It leads the Orange Pi by 13–20% on generation and 28–58% on prompt processing, and it gets the largest MTP gains.
2. **The Orange Pi Zero 3W comes close in a smaller, cheaper package.** Use `-t 2 -tb 8`, keep the fan on, and let the first boot finish.
3. **Compared with the Raspberry Pi 5, the UNO Q is 1.8–3.1× slower at generation and 10–14× slower at prompt processing.** Its A53 cores are compute-bound, so MTP never helps there. It runs every model up to 2B, including vision, but a caption takes 1.5–5 minutes.
4. **MTP pays off only on memory-bound boards, and only at n=3** (a verify batch of 4 tokens), for models of 2B and up.
5. **For agents and long contexts, the architecture matters more than the benchmark score.** MiniCPM5 2B has the highest Artificial Analysis score of the small models tested and the fastest short-context generation. But its generation speed collapses as the context grows. Qwen3.5 (hybrid) and Gemma 4 (sliding window) hold up much better.
6. **6 GB is the sweet spot for these boards.** 4 GB covers everything up to 2B with vision. 8 GB only helps to keep several models loaded or to use very long contexts: bigger models fit, but memory bandwidth makes them too slow for interactive use.

Benchmarks run and report drafted by Claude Opus 5.5 (Anthropic) under the direction of Marcelo Rovai. Raw data and scripts are available on request.